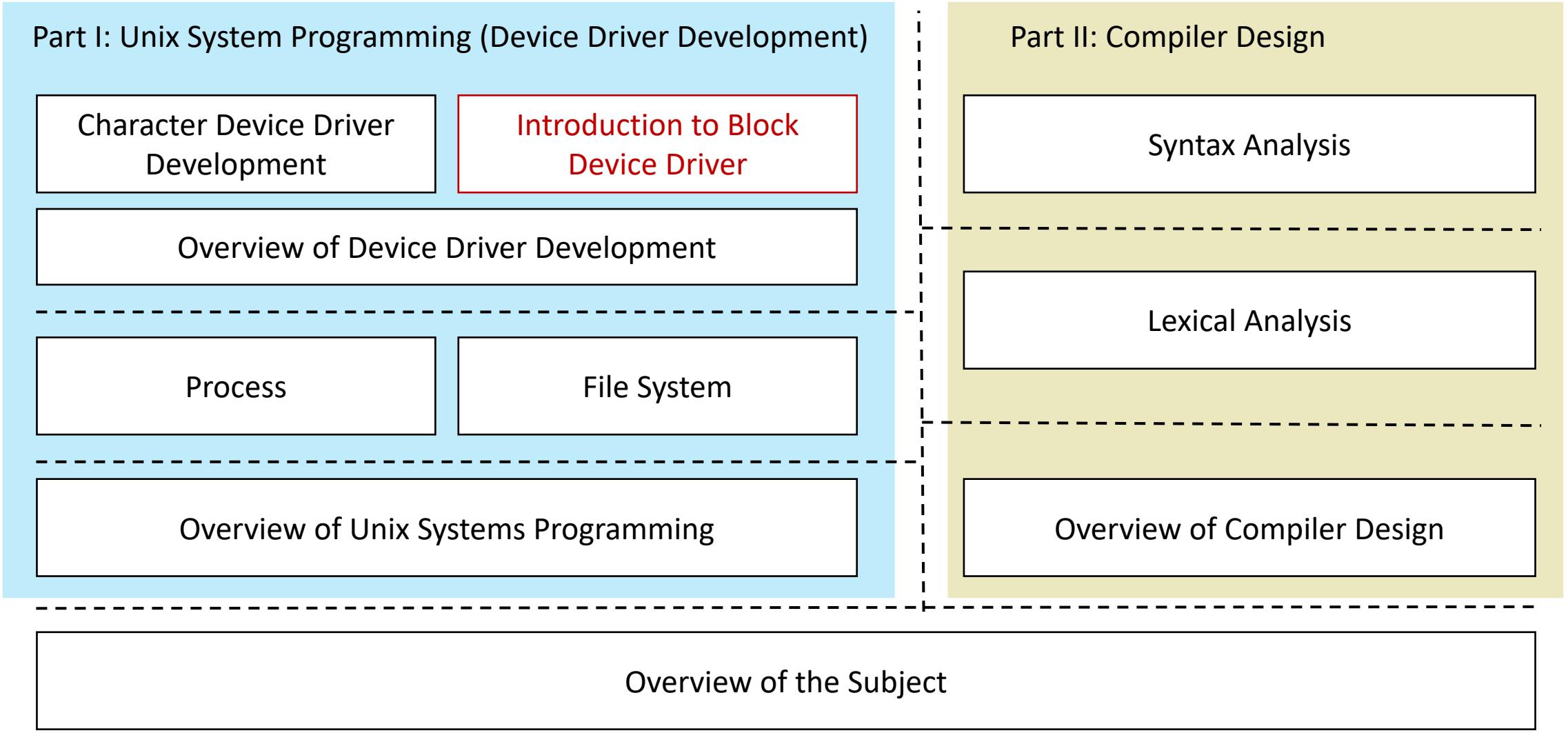
COMP3438 System Programming

Block Device Drivers

Course Organization



Outline

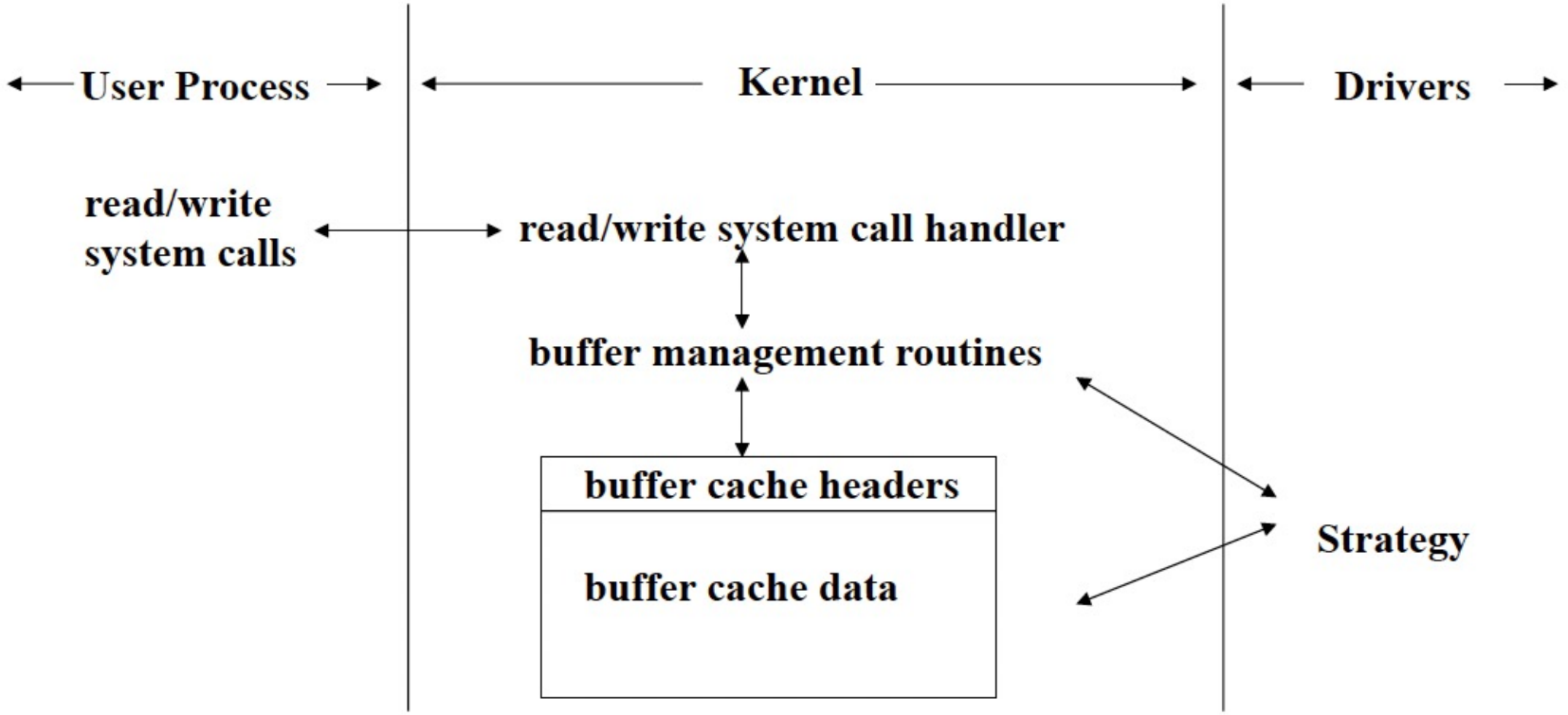
Introduction to Block Device Drivers

A Simple Block Device Driver – Sbull (Simple Block Utility for Loading Localities, see “Block Device Driver Demo Source Code”).

Block Device Drivers

Provide accesses to devices that transfer data in fixed-size blocks.

Communicate with OS through a collection of fixed size buffers.



A Simple Block Device Driver –Sbull (Simple Block Utility for Loading Localities)

A device driver for RAM disk: a segment of RAM is treated as storage.

A simplified vesion from the book: Linux Device Driver (chapter 12 loading device driver), 2nd edition, Alessandro Rubini, Jonathan Corbet.

http://www.xml.com/ldd/chapter/book/ch12.html

Main functions of a block driver

Part I: Functions related to `open`, `close`, `ioctl`, `check_media_change`, and `revalidate`, defined in `struct block_device_operations` in `<linux/blkdev.h>`

Part II: Functions related to requests (strategy)

Part III: Functions related to module management (load or unload).

Functions in `block_device_operations`

```
int sbull_open(struct block_device *device, fmode_t mode
  /*purpose of open: read, write, lseek etc.*/);
int sbull_release(struct gendisk *disk, fmode_t mode);
int sbull_media_changed(struct gendisk *gd);
int sbull_revalidate(struct gendisk *gd);
void sbull_invalidate(unsigned long ldev);
int sbull_ioctl(struct block_device *device, fmode_t mode, unsigned int cmd,
  unsigned long arg);


struct block_device_operatiions sbull_ops = {
   .owner = THIS_MODULE,
   .open = sbull_open,
   .release = sbull_release,
   .media_changed = sbull_media_changed,
   .revalidate_disk = sbull_revalidate,
   .ioctl = sbull_ioctl
};
```

Note there are no "`read`", "`write`," as they are handled by the kernel.

`Boldface`: will be reused.
`Red`: kernel function/structures
`Red`: important kernel functions/structures

Functions related to request (strategy)

```
static void sbull_transfer(struct sbull_dev *dev,
  unsigned long sector, unsigned long nsect,
  char *buffer, int write);


static void sbull_request(struct request_queue *q){
  struct request req;
  req = blk_fetch_request(q);
  ...
  sbull_transfer(dev, blk_rq_pos(req), blk_rq_cur_sectors(req),
    req->buffer, rq_data_dir(req));
  ...
}
```

Important data structure

```
struct sbull_dev{
  int size; //device size in sectors
  u8 *data; //the actual storage
  short users; //how many users
  short media_change; //flag a media change
  spinlock_t lock; //for mutual exclusion
  struct request_queue *queue; //the device request queue
  struct gendisk *gd; //the gendisk structure, profile of the
                      //device to be registered to kernel.
  struct timer_list timer; //for simulated media changes
};
```

Functions related to module management

```
static int __init sbull_init(void){
  ...
  sbull_major = register_blkdev(
    sbull_major, "sbull");
  ...
  setup_device(Devices + i, i);
  ...
}
```

```
static void setup_device(
  struct sbull_dev *dev, int which){
  ...
  dev->queue = blk_init_queue(
    sbull_request, &dev->lock);
  ...
  dev->gd = alloc_disk(SBULL_MINORS);
  ...
  dev->gd->fops = &sbull_ops;
  dev->gd->queue = dev->queue;
  ...
  add_disk(dev->gd);
  ...
}
```

Functions related to module management

```
static void sbull_exit(void){
   ...
   struct sbull_dev *dev = ...
   ...
   del_gendisk(dev->gd); //remove the gendisk and all its resources, deletes
                         //the partitions associated with the gendisk,
                         // unregisters the associated request_queue.
   put_disk(dev->gd); //decrements the gendisk refcount
   ...
   blk_cleanup_queue(dev->queue);
   ...
   unregister_blkdev(sbull_major, "sbull");
   ...
}
module_init(sbull_init);
module_exit(sbull_cleanup);
```

Summary

Block device drivers provide accesses to devices that transfer data in fixed-size blocks.

Bascially, there are three parts in a block device driver:

- Part I: Functions related to `open`, `close`, `ioctl`, `check_media_change`, and `revalidate`, defined in `struct block_device_operations` in `<linux/blkdev.h>`
- Part II: Functions related to requests (strategy)
- Part III: Functions related to module management (load or unload).

A simple block device driver is introduced.